# Programme de Cours: Base de Données et Data Science

Formation destinée aux étudiants de 3ème année (S7) de l'ENCG de Settat

12-14 semaines | 3 heures par semaine

**A. Larhlimi**



Année Académique 2026-2027

# Sommaire


1. Objectifs du cours
2. Tendances : en Data Science Financière
3. Structure du programme (14 semaines)
4. Outils et technologies
5. Aspects éthiques et réglementaires
6. Exemples pratiques
7. Méthodes d'évaluation
8. Conclusion

# Objectifs d'Apprentissage

- Maîtriser les concepts fondamentaux des bases de données relationnelles et non relationnelles
- Appliquer les techniques de Data Science à des problèmes financiers concrets
- Développer des compétences en analyse exploratoire, modélisation prédictive et visualisation
- Créer des rapports et tableaux de bord financiers interactifs avec Power BI
- Comprendre les enjeux éthiques et réglementaires de la Data Science en finance

# Tendances Data Science & Finance 2026

### IA agentic

Agents autonomes intelligents capables d'effectuer des tâches financières complexes sans intervention humaine

### Mesure des bénéfices GenAI

Évaluation rigoureuse du ROI et de la productivité réelle des outils d'IA générative

### Culture data-driven

Prise de conscience que la technologie seule ne suffit pas pour créer une véritable culture data

### Données non-structurées

Regain d'intérêt pour l'exploitation des données textuelles, images et documents non structurés

### Analyse en temps réel

Prise de décision financière accélérée grâce au traitement instantané des flux de données

### Sources alternatives

Exploitation de données satellites, réseaux sociaux et web pour enrichir l'analyse financière

# Aperçu de la Progression du Cours

**1-2 Fondamentaux des Bases de Données**
SQL, NoSQL, modèles relationnels

**3-4 Extraction et Nettoyage de Données**
APIs, Pandas, traitement des données

**5-6 Analyse Exploratoire et Visualisation**
Matplotlib/Seaborn, séries temporelles

**7-8 Modélisation Prédictive en Finance**
Machine learning, prédictions financières

**10 Données Non-Structurées et Alternatives**
Text mining, données alternatives

**11 Reporting avec Power BI**
Dashboards interactifs, visualisation avancée

**13 Blockchain & Aspects Réglementaires**
Smart contracts, RGPD, MiFID II

**14 Projets & Évaluation Finale**
Application intégrée des compétences acquises

**Organisation pédagogique**

Chaque semaine comprend une partie théorique (1h30) suivie d'une partie pratique sur ordinateur (1h30), permettant l'application immédiate des concepts. Des études de cas réels et des projets en groupe sont intégrés tout au long du programme pour renforcer l'apprentissage.

# Semaine 1-2 : Fondamentaux des Bases de Données



## Modèle relationnel : SQL

Structure des tables, clés primaires/étrangères, requêtes complexes (JOIN, GROUP BY), transactions ACID



## Introduction aux bases NoSQL

Bases documentaires (MongoDB), orientées colonnes (Cassandra), différences avec le modèle relationnel, théorème CAP



## Application à la finance

Modélisation d'un système de gestion de portefeuille, stockage de données de marché, comparaison SQL vs NoSQL pour cas d'usage financiers



## Pratique en laboratoire

Création de schémas de base de données sur Jupyter/Google Colab, connexion à SQLite et MongoDB, premières requêtes sur données financières

# Semaine 3-4 : Extraction et Nettoyage de Données Financières

### Collecte des données de marché
APIs financières, fichiers CSV/Excel, web scraping

### Manipulation avec Pandas
DataFrame, Series, opérations vectorisées

### Nettoyage et transformation
Gestion des valeurs manquantes, outliers et erreurs

### TP pratique
Importation et nettoyage d'un dataset boursier réel

```python
# Exemple d'importation avec Pandas
import pandas as pd
import yfinance as yf

data = yf.download('AAPL',
  start='2020-01-01',
  end='2025-01-01')
```

```python
# Nettoyage des données
data.fillna(method='ffill', inplace=True)
data['Returns'] = data['Close'].pct_change()
data = data[data['Volume'] > 0]
data = data[~data['Returns'].isin([float('inf'),
float('-inf')])]
```

**Points clés à maîtriser :**

- Connaissance des principales sources de données financières
- Manipulation efficace des séries temporelles avec Pandas
- Détection et traitement des anomalies dans les données boursières

# Semaine 5-6 : Analyse Exploratoire & Visualisation de Données

### Statistiques descriptives

Mesures de tendance centrale, dispersion, distributions. Application aux indicateurs financiers clés.

### Introduction à Matplotlib/Seaborn

Création de visualisations statiques: histogrammes, box plots, scatter plots, heatmaps pour l'analyse de corrélation.

### Séries temporelles financières

Analyse des tendances, saisonnalité, autocorrélation. Traitement des données boursières et économiques.

### Visualisation dynamique dans Power BI

Construction de dashboards interactifs, filtres croisés, visualisations avancées pour l'analyse financière.

Exemple de visualisation financière



# Semaine 7-8 : Modélisation Prédictive en Finance

### Régression linéaire et logistique

Applications aux prévisions de cours et analyse des rendements financiers

### Applications : scoring crédit et prévision

Modélisation du risque de crédit et prédiction des tendances de marché

### Introduction au machine learning

Arbres de décision, forêts aléatoires et SVM pour l'analyse financière

### Pratique avec Scikit-learn

Implémentation des modèles et évaluation des performances

Exemple : Modèle de prédiction des cours boursiers




**Points clés :**

Compréhension des algorithmes de machine learning adaptés aux données financières et capacité à évaluer la pertinence des modèles pour différentes applications de trading et gestion des risques.

# Semaine 9-10 : Données Non-Structurées et Alternatives

## Text Mining

- Extraction d'informations des actualités financières
- Analyse de sentiment sur les réseaux sociaux
- Techniques NLP appliquées aux rapports financiers
- Prédiction de volatilité basée sur les news

## Données Alternatives

- Données satellites (trafic, occupation parkings)
- Données web (trafic site, comportement utilisateurs)
- Exploitation des données de paiement anonymisées
- Réglementation sur les données alternatives

## RAG et Embeddings Financiers

- Introduction aux méthodes RAG pour la finance
- Création d'embeddings de documents financiers
- Recherche sémantique dans les rapports annuels
- Chatbots financiers augmentés par RAG

## Étude de Cas

Impact des actualités sur les marchés financiers :

- Analyse de l'effet des annonces de résultats
- Mesure de l'impact des événements économiques
- Construction d'un indicateur de sentiment marché

## TP & Projet

Construction d'un modèle prédictif basé sur l'analyse de sentiment des actualités financières.

# Semaine 11 : Reporting & Tableaux de Bord avec Power BI

## Principes du reporting financier interactif

- Définition d'indicateurs clés de performance financière
- Structuration des données pour un reporting efficace
- Conception adaptée aux besoins des différentes parties prenantes

## Construction d'un dashboard Power BI

- Import et modélisation des données financières
- Création de mesures et colonnes calculées (DAX)
- Conception de visualisations interactives efficaces




*TP : Création d'un tableau de bord de suivi de portefeuille*

## Publication et partage sécurisé des analyses

- Paramètres de sécurité et confidentialité des données
- Partage ciblé et gestion des accès utilisateurs
- Mise en place de rafraîchissements automatiques

# Semaine 12 : Blockchain & Sécurité des Données

### Introduction à la blockchain

Comprendre les principes fondamentaux : décentralisation, immutabilité et consensus. Application des structures de données distribuées en finance.

### Smart contracts en finance

Automatisation des contrats et exécution autonome. Cas d'usage : règlement automatique de dérivés, KYC partagé et gestion des prêts décentralisés (DeFi).

### Sécurité, cryptographie et cas d'usage réels

Protection des données financières par cryptographie. Applications actuelles : tokenisation d'actifs, transactions transfrontalières, et traçabilité des titres financiers.

TP : Analyse d'un smart contract simple et simulation de son exécution dans un environnement sandbox

# Semaine 13 : Aspects Éthiques & Réglementaires



## RGPD vs. MiFID II

Équilibre entre protection des données personnelles et exigences KYC/transparence des marchés. Défis de conformité pour les institutions financières.



## Éthique algorithmique, biais et transparence

Détection et atténuation des biais dans les modèles financiers. Explicabilité des décisions algorithmiques et transparence envers les clients.



## Limites de l'IA en finance – risques et responsabilités

Gouvernance des algorithmes et responsabilité humaine. Risques systémiques et cadres de supervision des systèmes automatisés.



## Cadre d'évaluation éthique des projets Data Science

Méthodologie d'évaluation des implications éthiques. Checklist de conformité pour les projets d'analyse de données financières.

# Semaine 14 : Projets Pratiques & Évaluation



## Présentation de projets de groupe

- Sessions de 20 minutes par équipe
- Démonstration de solution complète
- Q&A avec l'audience et les formateurs



## Critères d'évaluation

- Qualité technique (40%)
- Pertinence business (30%)
- Clarté de la présentation (20%)
- Documentation et reproductibilité (10%)



## Mise en application de bout en bout

- Extraction et traitement de données réelles
- Application d'algorithmes prédictifs
- Visualisation interactive des résultats



## Exemples de sujets

- Système de scoring crédit avec alternatives data
- Dashboard d'analyse de performance de portefeuille
- Détection d'anomalies pour la lutte anti-fraude

# Conclusion & Questions

## Vers une finance augmentée par la donnée

À l'horizon 2025-2030, les professionnels de la finance qui maîtriseront la data science seront au cœur de la transformation du secteur financier :

- Prise de décision basée sur l'analyse prédictive et en temps réel
- Détection avancée des risques et des opportunités grâce à l'IA
- Innovation dans les services financiers via les données alternatives

## Vous avez des questions?

Ce programme vous aidera à acquérir les compétences techniques et conceptuelles pour devenir un professionnel de la finance à l'ère des données.




Pour plus d'informations sur ce programme, contactez-nous :

Abderrahim.larhlimi@uhp.ac.ma

# Sciences des données appliquées à la finance et au commerce international

Des données aux décisions pour des marchés plus performants, plus résilients et plus durables

## 1 Comprendre les concepts de base de la data science

- Qu'est-ce que la data science ?
- Données, information et connaissance
- Statistiques, IA, machine learning, deep learning
- Types et sources de données
- Outils et langages (Python, R, SQL, etc.)
- Ethique et qualité des données

Acquérir une base solide pour analyser des données réelles

## 2 Maîtriser les techniques de la fouille de données

- Prétraitement et nettoyage des données
- Analyse exploratoire et visualisation
- Classification et régression
- Clustering et segmentation
- Règles d'association et analyse de paniers
- Détection d'anomalies
- Séries temporelles et prévisions
- Text mining et traitement du langage naturel

Extraire des connaissances utiles à partir de grands volumes de données

## 3 Applications en finance

- Analyse et prévision des marchés financiers
- Gestion des risques (crédit, marché, liquidité)
- Détection de fraudes et de comportements suspects
- Scoring de crédit et analyse de solvabilité
- Gestion de portefeuille et allocation d'actifs
- Analyse de sentiment sur les actualités financières
- Finance durable (ESG)



**Exemple :**
Prédire le risque de défaut d'une entreprise à partir de ses données financières.

## 4 Applications au commerce international

- Prévision des échanges et des flux commerciaux
- Analyse des tendances et de la demande mondiale
- Détection des risques pays et des instabilités géopolitiques
- Optimisation des chaînes logistiques et des routes maritimes
- Analyse des coûts et des délais de transport
- Segmentation des marchés et identification d'opportunités
- Veille concurrentielle et analyse des textes réglementaires



**Exemple :**
Anticiper l'évolution des exportations d'un produit vers différents pays.

## 5 Méthodologie et mise en œuvre

- Définition du problème métier
- Collecte et intégration de données multi-sources
- Préparation et transformation des données
- Choix et entraînement des modèles
- Évaluation et interprétation des résultats
- Déploiement et suivi des modèles
- Communication des insights aux décideurs




## 6 Compétences développées et bénéfices

**Compétences clés**

- Analyse et modélisation de données complexes
- Utilisation d'outils et d'algorithmes avancés
- Interprétation des résultats dans un contexte financier et international
- Traduction des analyses en recommandations concrètes

**Bénéfices attendus**

- Meilleure prise de décision
- Réduction des risques
- Optimisation des performances
- Avantage concurrentiel durable
- Contribution à des marchés plus transparents et inclusifs

**Objectif final :** Appliquer les connaissances des sciences de données aux domaines de la finance et du commerce international pour relever des défis réels et créer de la valeur économique et sociale.

**A. Larhlimi**
Sciences des données | Finance | Commerce international

CREATED BY: Your Name DATE: septembre-2025

# Bases de Données et Data Science



Exploration des fondamentaux et de l'impact de la Data Science.

# Introduction aux Bases de Données

## Définition

Les bases de données sont des systèmes structurés conçus pour recueillir, stocker et gérer des données de manière organisée et accessible, facilitant ainsi leur utilisation.



# Types de Bases de Données

Les bases de données non relationnelles stockent des données diverses telles que des documents ou des paires clé-valeur. Des exemples majeurs sont MongoDB et Cassandra, qui offrent flexibilité et évolutivité.

## Bases Non Relationnelles

Les bases de données relationnelles structurent les données sous forme de tables. Elles permettent d'effectuer des requêtes complexes via SQL. Exemples notables incluent MySQL et PostgreSQL.

## Bases Relationnelles

# Processus de Data Science

Rassembler des données pertinentes à partir de divers sources comme des bases de données ou des API.

## Collecte

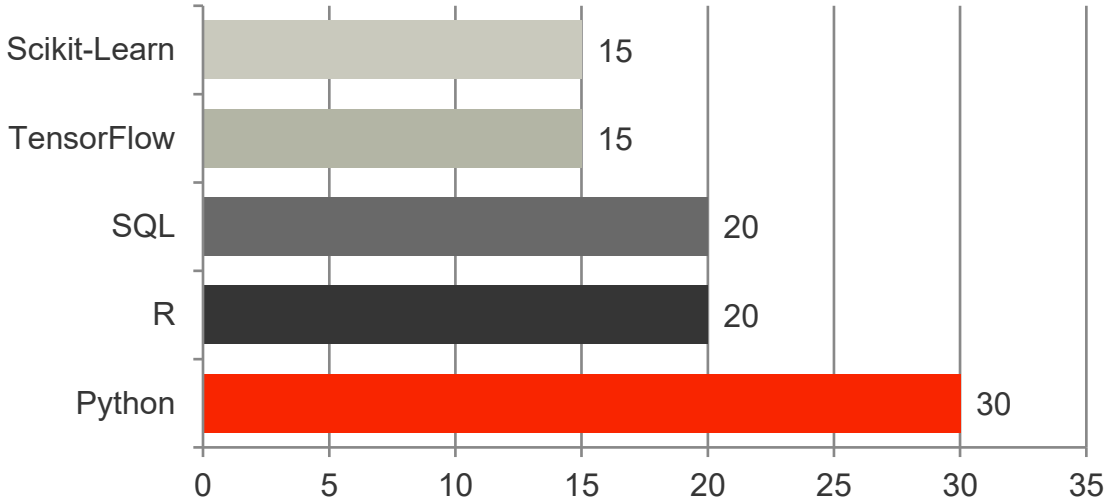
Préparer les données en éliminant duplications et valeurs manquantes pour garantir leur qualité.

## Nettoyage

Créer des modèles algorithmiques visant à extraire des informations significatives des données.

## Modélisation

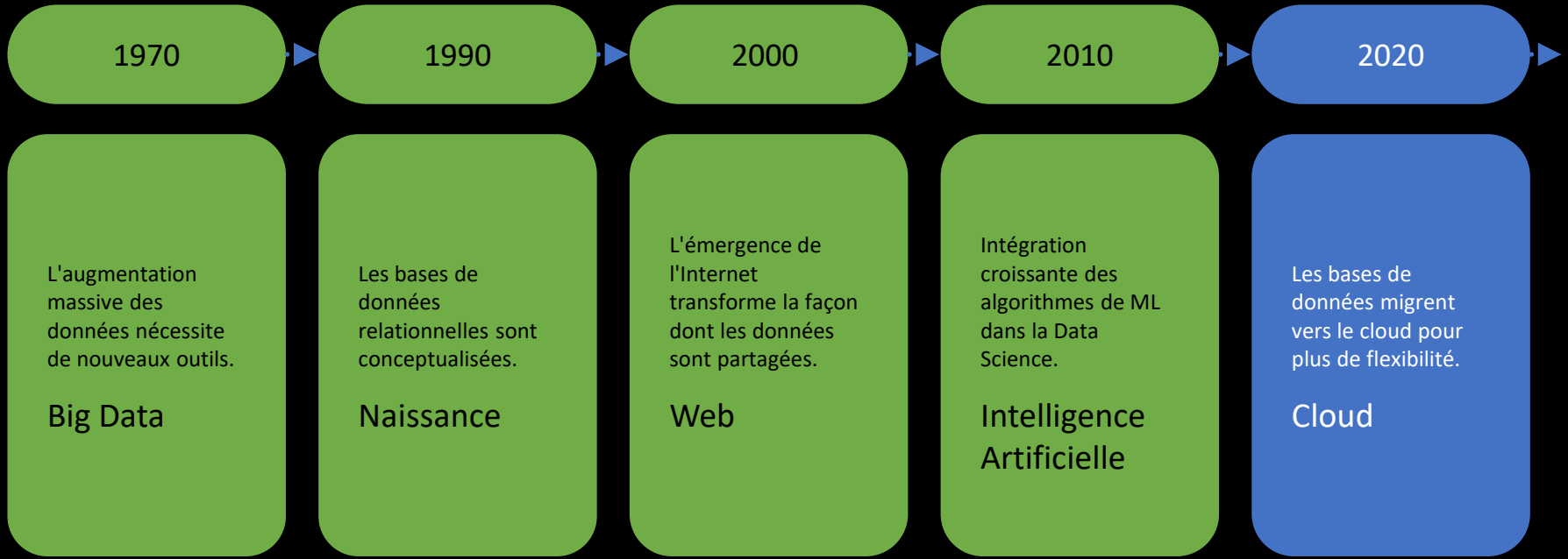
# Techniques et Outils




Python est le principal outil avec 30 % d'utilisation, suivi de R (20 %) et SQL (20 %). TensorFlow et Scikit-Learn représentent chacun 15 %, facilitant des analyses avancées.

## Répartition des outils

# Évolution du Domaine



# Applications et Perspectives




L'évolution de la Data Science sera fortement influencée par les avancées en intelligence artificielle, augmentant son rôle dans l'analyse et la prise de décision stratégique.

## Avenir de Data Science

La Data Science s'applique dans plusieurs secteurs principaux tels que la santé, la finance, et le marketing, où elle aide à optimiser les processus et à prendre des décisions basées sur des données.

## Domaines d'Application

# Gestion des bases de données

Concepts fondamentaux, modèle relationnel, sécurité et SQL



# Objectifs du cours

1. Comprendre les fondamentaux
2. Modéliser une base de données
3. Gérer une base de données
4. Sécuriser les données
5. Interroger avec SQL
6. Exploiter les données

# Pourquoi une base de données ?

## Scenario : Gestion universitaire

- 10 000+ étudiants
- 500+ enseignants
- 1000+ cours
- 50 000+ inscriptions
- 300 000+ notes

**Question : Peut-on gérer ces informations avec plusieurs fichiers Excel indépendants ?**

# Donnée → Information




**Exemple : "ALI" (donnée brute) → analyse → "nom d'étudiant" (information)**

Une base de données organise et structure les données pour en extraire de l'information pertinente.

# Définition d'une base de données

**Ensemble structuré de données, organisé pour permettre un accès, une mise à jour et une interrogation efficace.**

1. Structurée : organisée selon un schéma défini
2. Persistante : les données survivent à l'arrêt de l'application
3. Cohérente : respecte des règles d'intégrité
4. Partagée : accessible par plusieurs utilisateurs
5. Interrogeable : requêtes complexes possibles

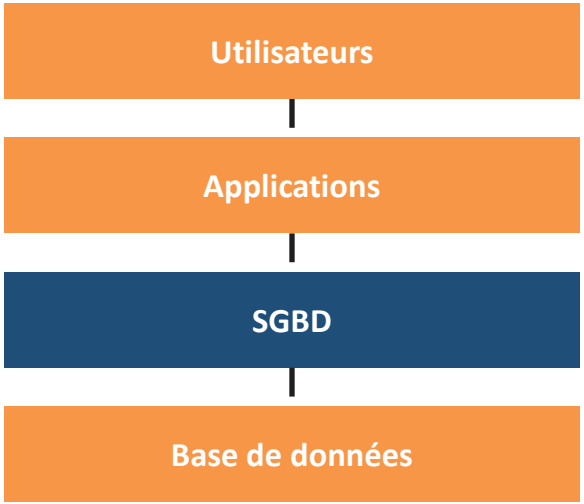
# Fichiers classiques vs Base de données

| Critère | Fichiers indépendants | Base de données |
|---|---|---|
| Redondance | Élevée (risque d'incohérence) | Minimale (contrôlée) |
| Cohérence | Difficile à assurer | Garantie par SGBD |
| Recherche | Lente, coûteuse | Rapide, optimisée |
| Sécurité | Limitée | Complète (authentif., chiffrement) |
| Accès simultané | Problématique | Géré par SGBD |
| Maintenance | Complexe | Centralisée |

# Qu'est-ce qu'un SGBD ?

## Système de Gestion de Base de Données (SGBD)

*Logiciel qui gère l'accès, la modification, la sécurité et l'intégrité d'une base de données.*



# Fonctions principales d'un SGBD

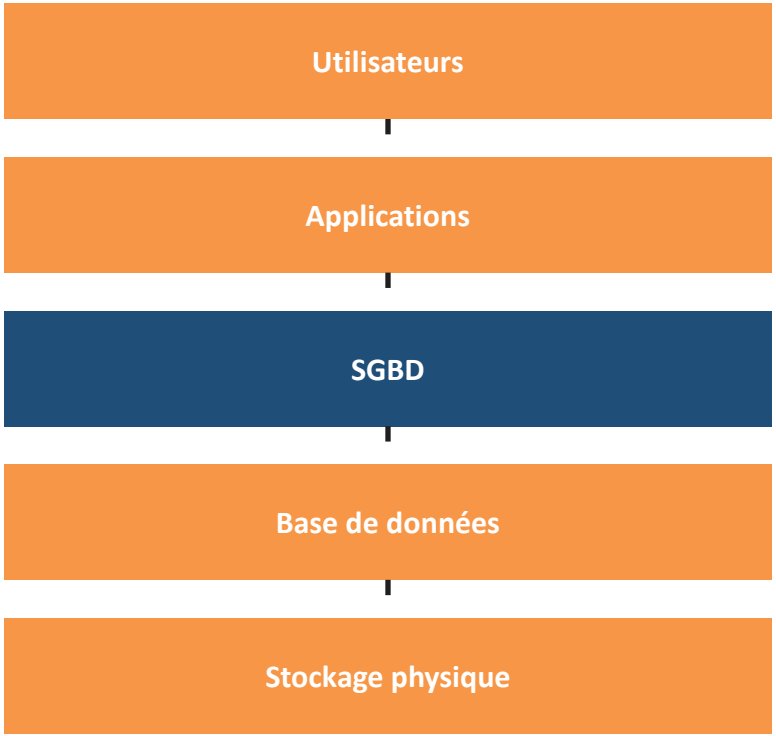
| | |
|---|---|
| **Stockage** | Gère le stockage physique des données |
| **Interrogation** | Permet des requêtes complexes (SQL) |
| **Modification** | INSERT, UPDATE, DELETE contrôlés |
| **Intégrité** | Vérifie les contraintes (clés, domaines) |
| **Sécurité** | Authentification et contrôle d'accès |
| **Concurrence** | Gère plusieurs utilisateurs simultanés |
| **Sauvegarde** | Backup automatique et restauration |
| **Récupération** | Reprise sur erreur ou panne |

# Exemples de SGBD du marché

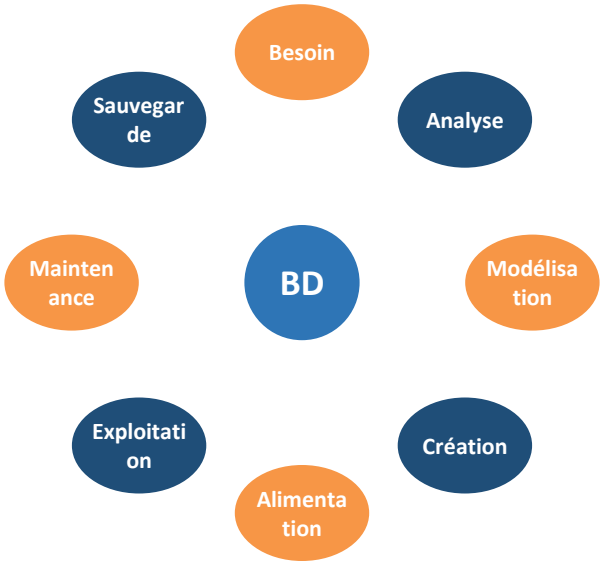
| | |
|---|---|
| **PostgreSQL** | Open source, puissant, gratuit |
| **MySQL/MariaDB** | Web, populaire, léger |
| **SQLite** | Embarqué, petit, pas réseau |
| **Oracle Database** | Entreprise, cher, robuste |
| **Microsoft SQL Server** | Windows, intégration MS Office |
| **MongoDB** | NoSQL, documents, flexibilité |

**Ce cours utilise PostgreSQL (open source, pédagogique, professionnel)**

# Architecture d'un système de base de données



# Cycle de vie d'une base de données



# Gestion d'une base de données

## Rôle de l'administrateur de base de données (DBA)

- Création et maintenance des tables et index
- Gestion des utilisateurs et des permissions
- Sauvegarde régulière et restauration
- Optimisation des performances
- Assurer la disponibilité (24h/24)
- Mise à jour et mises à jour du SGBD
- Monitoring et alertes
- Documentation et procédures

# Pourquoi sécuriser les données ?

## Scenario : Gestion académique

**Question : Tous les employés d'une université doivent-ils pouvoir modifier les notes des étudiants ?**

Réponse : Non ! Seuls les enseignants de chaque cours peuvent modifier leurs notes.

**Confidentialité**

Seuls les autorisés accèdent

**Intégrité**

Les données ne sont pas corrompues

**Disponibilité**

# Authentification vs Autorisation

### Authentification

**Qui êtes-vous ?**

Vérifier l'identité de l'utilisateur (login + mot de passe ou clé)

### Autorisation

**Que pouvez-vous faire ?**

Attribuer des permissions (SELECT, INSERT, UPDATE, DELETE) selon le rôle

**Exemple : Enseignant**

1. Authentification : L'université vérife qu'il est bien enseignant (mot de passe)

2. Autorisation : L'enseignant peut SELECT/UPDATE uniquement les notes de ses cours

3. Restriction : L'enseignant ne peut pas DELETE, ne peut pas voir d'autres données

# Utilisateurs, rôles et privilèges

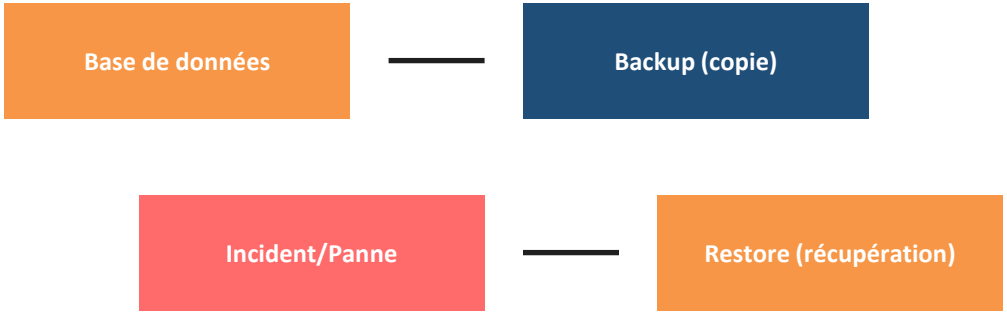
| | |
|---|---|
| **Administrateur** | SELECT • INSERT • UPDATE • DELETE • CREATE • DROP |
| **Enseignant** | SELECT • UPDATE (notes) |
| **Étudiant** | SELECT (leurs notes) |

### Matrice des droits d'accès

| Opération | Admin | Enseignant | Étudiant |
|---|---|---|---|
| SELECT | ✓ | ✓ | ✓ (limité) |
| INSERT | ✓ | ✗ | ✗ |
| UPDATE | ✓ | ✓ (leurs cours) | ✗ |
| DELETE | ✓ | ✗ | ✗ |
| CREATE TABLE | ✓ | ✗ | ✗ |

# Sauvegarde et restauration

## Protection contre les incidents




## Stratégies courantes

- Backup quotidien : chaque nuit à 23h
- Backup horaire : toutes les heures durant les heures de bureau
- Sauvegarde sur site (local) + hors site (cloud)
- RPO (Recovery Point Objective) : données max 1h perdue acceptables ?
- RTO (Recovery Time Objective) : combien de temps pour restaurer ?

# Exercice : Sécurité universitaire

## Scenario

Une université possède une base de données académique avec les tables suivantes :

- Administrateur (gestionnaire IT)
- Enseignant (consultant les notes et les met à jour)
- Étudiant (consulte uniquement ses notes)
- Service de scolarité (gère les inscriptions)

**Travail en petits groupes (15 minutes)**

1. Déterminer les permissions (SELECT, INSERT, UPDATE, DELETE) pour chaque rôle
2. Définir les restrictions (exemple : enseignant = ses cours uniquement)
3. Identifier les données sensibles (exemple : notes)

# Introduction au modèle relationnel

## Concept fondamental

**Une base relationnelle organise les données sous forme de relations, représentées par des tables.**

### Exemple : Table ETUDIANT

| id | nom | prenom | email |
|---|---|---|---|
| 1 | Dupont | Jean | jean.dupont@univ.ca |
| 2 | Martin | Marie | marie.martin@univ.ca |
| 3 | Bernard | Pierre | pierre.bernard@univ.ca |

Chaque ligne = un étudiant. Chaque colonne = une propriété. Chaque cellule = une valeur.

# Terminologie : Table, ligne, colonne

Attribut (colonne)

| id_etudiant | nom | prenom | email |
|---|---|---|---|
| 1 | Dupont | Jean | jean.dupont@univ.ca |
| 2 | Martin | Marie | marie.martin@univ.ca |

Tuple (ligne)

**Vocabulaire clé**

- Table : ensemble d'étudiants
- Attribut (colonne) : propriété (id, nom, prenom, email)
- Tuple (ligne) : un enregistrement (un étudiant)
- Domaine : type de données (INT, VARCHAR, DATE)
- Cardinalité : nombre de tuples dans la table
- Arité : nombre d'attributs (ici 4)

# Clé primaire

## PRIMARY KEY (Clé primaire)

**Attribut (ou combinaison d'attributs) qui identifie de manière unique chaque tuple dans une table.**

### Exemple : ETUDIANT

| PK | id_etudiant | nom | email |
|---|---|---|---|
| | 1 | Dupont | jean.dupont@univ.ca |
| | 2 | Martin | marie.martin@univ.ca |

### Propriétés d'une clé primaire

- Unicité : chaque id_etudiant apparaît une seule fois
- Non-nullité : chaque étudiant doit avoir un id_etudiant
- Stabilité : la clé ne change pas au fil du temps
- Minimalité : pas d'attribut superflu dans la clé

# Clé étrangère

## FOREIGN KEY (Clé étrangère)

**Attribut qui référence la clé primaire d'une autre table. Elle crée une relation entre deux tables.**

### Table ETUDIANT

| PK | id_etudiant | nom |
|---|---|---|
| | 1 | Dupont |
| | 2 | Martin |

### Table INSCRIPTION

| PK | id_etudiant | FK | id_cours |
|---|---|---|---|
| | 1 | | 101 |
| | 2 | | 102 |

**Interprétation :**

- id_étudiant dans INSCRIPTION est une clé étrangère
- Elle référence id_étudiant dans ETUDIANT
- Chaque inscription est liée à un étudiant réel
- On ne peut pas inscrire un étudiant qui n'existe pas (intégrité référentielle)
- Si on supprime un étudiant, ses inscriptions doivent être gérées (DELETE CASCADE ou SET NULL)

# Relations et cardinalités

## Trois types de relations

### 1:1 (Un-à-Un)

Un étudiant = un passeport
Un employé = un bureau

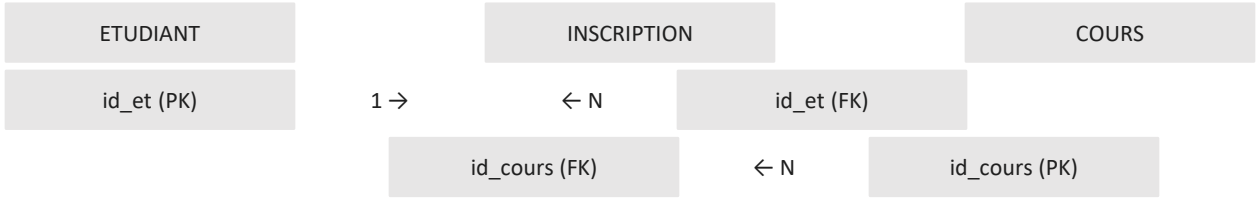
### 1:N (Un-à-Plusieurs)

Un enseignant = plusieurs cours
Un étudiant = plusieurs inscriptions

### N:M (Plusieurs-à-Plusieurs)

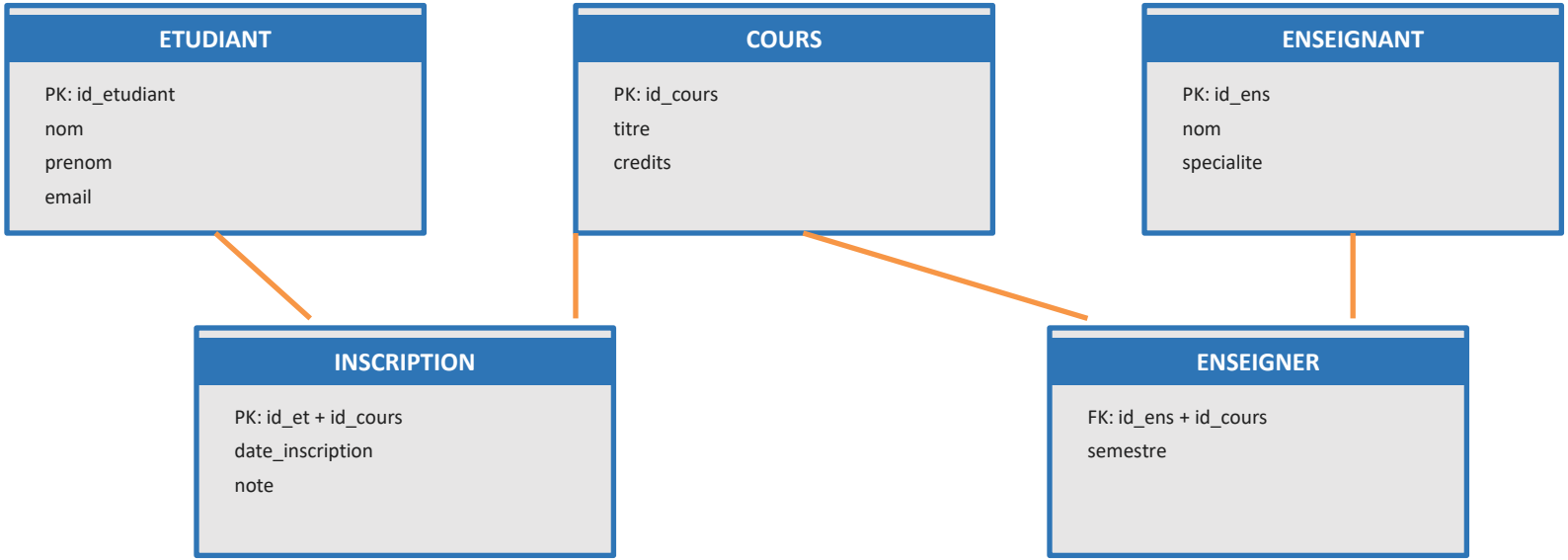
Un étudiant = plusieurs cours
Un cours = plusieurs étudiants

## Représentation N:M (table d'association)



# Schéma relationnel : Base universitaire

## Modèle des données universitaires




PK = Clé primaire | FK = Clé étrangère | Lignes = Relations

# Intégrité des données

### Intégrité de domaine

Les valeurs sont du bon type et dans le bon format

*Exemple : age doit être un entier entre 0 et 150*

### Intégrité d'entité

Chaque tuple est unique (clé primaire non-null)

*Exemple : id_etudiant doit être unique et non-null*

### Intégrité référentielle

Les clés étrangères doivent référencer des tuples existants

*Exemple : id_etudiant dans INSCRIPTION doit exister dans ETUDIANT*

# Exercice : Modéliser une base de données

## Scenario

Une bibliothèque souhaite gérer :

- Livres (titre, auteur, ISBN, année)
- Auteurs (nom, prénom, nationalité)
- Adhérents (nom, email, téléphone)
- Emprunts (date d'emprunt, date de retour, amende)

**Travail en petits groupes (20 minutes)**

1. Identifier les entités (tables)
2. Lister les attributs de chaque entité
3. Choisir la clé primaire de chaque table
4. Identifier les relations (1:1, 1:N, N:M)
5. Dessiner le schéma relationnel
6. Définir les clés étrangères

# Introduction à SQL

## SQL = Structured Query Language

**Langage déclaratif standardisé pour interroger et modifier les bases de données.**

| | | |
|---|---|---|
| **DDL** | Data Definition Language | CREATE, DROP, ALTER |
| **DML** | Data Manipulation Language | INSERT, UPDATE, DELETE |
| **DQL** | Data Query Language | SELECT |
| **DCL** | Data Control Language | GRANT, REVOKE |
| **TCL** | Transaction Control Language | COMMIT, ROLLBACK |

# CREATE TABLE : créer une table

### Syntaxe

```
CREATE TABLE Etudiant (
    id_etudiant INT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    prenom VARCHAR(100),
    email VARCHAR(150) UNIQUE,
    date_naissance DATE
);
```

### Explication

- INT : type entier
- VARCHAR(100) : texte jusqu'à 100 caractères
- PRIMARY KEY : clé primaire, unique et non-null
- NOT NULL : la valeur est obligatoire
- UNIQUE : chaque valeur doit être différente
- DATE : type date

# INSERT : insérer des données

### Syntaxe

```
INSERT INTO Etudiant (id_etudiant, nom, prenom, email)
VALUES (1, 'Dupont', 'Jean', 'jean@univ.ca');

INSERT INTO Etudiant VALUES (2, 'Martin', 'Marie', 'marie@univ.ca', '2000-05-15');

INSERT INTO Etudiant (id_etudiant, nom, prenom)
VALUES (3, 'Bernard', 'Pierre');
```

### Remarques

- Première syntaxe : indiquer les colonnes
- Deuxième syntaxe : toutes les colonnes dans l'ordre
- Troisième syntaxe : certaines colonnes seulement (email nullable)
- Les chaînes de caractères sont entre guillemets simples (')
- Chaque INSERT crée une nouvelle ligne

# SELECT : interroger les données

### Syntaxe basique

```
SELECT * FROM Etudiant;

SELECT nom, prenom FROM Etudiant;

SELECT DISTINCT prenom FROM Etudiant;
```

### Résultats

| Requête | Résultat |
| --- | --- |
| SELECT * | Toutes les colonnes, toutes les lignes |
| SELECT nom, prenom | Seulement nom et prenom |
| SELECT DISTINCT | Élimine les doublons |

# WHERE : filtrer les données

### Syntaxe

```
SELECT nom, prenom FROM Etudiant WHERE id_etudiant = 1;

SELECT nom FROM Etudiant WHERE nom <> 'Dupont';

SELECT * FROM Etudiant WHERE id_etudiant > 5 AND nom = 'Martin';
```

### Opérateurs de comparaison

- = (égal)
- <> (différent)
- > (plus grand)
- < (plus petit)
- >= (≥)
- <= (≤)
- AND, OR, NOT

**Logique : WHERE colonne opérateur valeur [AND/OR condition]**

# ORDER BY et DISTINCT

## ORDER BY : trier les résultats

```
SELECT nom, prenom FROM Etudiant ORDER BY nom ASC;
SELECT nom FROM Etudiant ORDER BY id_etudiant DESC;
```

*ASC (ascending) = A-Z, 0-9. DESC (descending) = Z-A, 9-0*

## DISTINCT vs WHERE

| Concept | Définition | Exemple |
|---|---|---|
| WHERE | Filtre les lignes avant sélection | WHERE nom = 'Dupont' : seulement Dupont |
| ORDER BY | Trie les lignes après sélection | ORDER BY nom ASC : A-Z |
| DISTINCT | Élimine les doublons | DISTINCT prenom : chaque prenom une fois |

# Fonctions d'agrégation

## Calculer des statistiques

```
SELECT COUNT(*) FROM Etudiant;
SELECT AVG(note) FROM Inscription;
SELECT MAX(id_etudiant) FROM Etudiant;
SELECT SUM(credits) FROM Cours;
```

## Fonctions disponibles

| Fonction | Signification | Exemple |
|---|---|---|
| COUNT(...) | Compte les lignes | COUNT(*) → 100 |
| SUM(...) | Somme les valeurs | SUM(credits) → 120 |
| AVG(...) | Moyenne des valeurs | AVG(note) → 78.5 |
| MIN(...) | Valeur minimale | MIN(id) → 1 |
| MAX(...) | Valeur maximale | MAX(note) → 95 |

# GROUP BY et HAVING

## Grouper et filtrer les agrégations

```
SELECT id_cours, COUNT(*) FROM Inscription GROUP BY id_cours;

SELECT id_cours, AVG(note) FROM Inscription
GROUP BY id_cours HAVING AVG(note) > 75;
```

## Différence : WHERE vs HAVING

| | | |
|---|---|---|
| **WHERE** | Filtre les lignes AVANT agrégation | *WHERE id_cours = 101* |
| Filtre les lignes AVANT agrégation | | |
| **HAVING** | Filtre les groupes APRÈS agrégation | *HAVING COUNT(*) > 5* |
| Filtre les groupes APRÈS agrégation | | |

# Jointures : combiner des tables

## Lier deux tables

**ETUDIANT**

| id | nom |
|---|---|
| 1 | Dupont |
| 2 | Martin |

Lien : INSCRIPTION.id_et → ETUDIANT.id

**INSCRIPTION**

| id_et | id_cours | note |
|---|---|---|
| 1 | 101 | 85 |
| 1 | 102 | 90 |
| 2 | 101 | 78 |

## Types de jointures

| | |
|---|---|
| **INNER JOIN** | Seulement les lignes correspondantes |
| **LEFT JOIN** | Toutes les lignes de gauche + correspondances |
| **RIGHT JOIN** | Toutes les lignes de droite + correspondances |
| **FULL JOIN** | Toutes les lignes des deux côtés |

# Requête multi-tables : exemple complet

### Question : "Afficher le nom de chaque étudiant avec les cours auxquels il est inscrit"

```
SELECT e.nom, e.prenom, c.titre
FROM Etudiant e
INNER JOIN Inscription i ON e.id = i.id_etudiant
INNER JOIN Cours c ON i.id_cours = c.id_cours
ORDER BY e.nom;
```

### Résultat attendu

| nom | prenom | titre |
|---|---|---|
| Dupont | Jean | Mathématiques I |
| Dupont | Jean | Français II |
| Martin | Marie | Mathématiques I |

*Note : Jean Dupont apparaît deux fois car il est inscrit à deux cours*

# Exercices SQL - Niveau 1 (débutant)

**Thème : Gestion universitaire**

À votre tour ! Écrivez les requêtes suivantes :

1. Afficher tous les étudiants (id, nom, prenom)
2. Afficher uniquement les noms des étudiants
3. Afficher les étudiants dont id > 5
4. Afficher les étudiants triés par nom A→Z
5. Afficher les étudiants avec email unique
6. Afficher les 10 premiers étudiants

**Solutions pas affichées sur la slide.**

Les solutions détaillées se trouvent dans les notes du présentateur (page suivante).

*Ces exercices travaillent : SELECT, FROM, WHERE, ORDER BY, DISTINCT*

# Exercices SQL - Niveau 2 (intermédiaire)

**Thème : Gestion universitaire**

À votre tour ! Écrivez les requêtes suivantes :

1. Compter le nombre total d'étudiants
2. Afficher la moyenne des notes
3. Afficher le nombre d'inscriptions par étudiant
4. Afficher les cours avec plus de 20 étudiants inscrits
5. Afficher le nom et prénom de chaque étudiant avec le titre de son cours
6. Afficher les étudiants qui n'ont pas d'inscription

**Solutions pas affichées sur la slide.**

Les solutions détaillées se trouvent dans les notes du présentateur.

*Ces exercices travaillent : COUNT, AVG, GROUP BY, HAVING, JOIN, LEFT JOIN*

# Exercices SQL - Niveau 3 (avancé)

**Thème : Gestion universitaire**

À votre tour ! Écrivez les requêtes suivantes :

1. Identifier les étudiants inscrits à PLUS DE 3 COURS et calculer leur moyenne générale
2. Afficher le nom du cours, le nombre d'étudiants inscrits et la note moyenne par cours
3. Afficher les étudiants qui ont une moyenne ≥ 80 dans TOUS leurs cours
4. Lister les enseignants et le nombre de cours différents qu'ils enseignent
5. Identifier les étudiants qui n'ont aucune inscription et les lister séparément

**Niveau de réflexion :Ces exercices demandent de combiner plusieurs notions et de raisonner avant d'écrire SQL.**

Solutions pas affichées sur la slide. Les corrections se trouvent dans les notes du présentateur.

# Mise en situation intégrative

## Mini-projet : Construire un système académique

### Scénario

Une école souhaite construire son système de gestion académique. Elle a besoin de gérer :

- Les étudiants (identifiant unique, nom, email)
- Les cours (titre, code, crédits)
- Les inscriptions (étudiant → cours, date, note)
- Les enseignants (nom, spécialité)
- Les relations enseignant ↔ cours

### Tâches à réaliser (travail en groupes, 45 minutes)

1. Identifier les entités et proposer un schéma relationnel
2. Créer les 5 tables avec CREATE TABLE
3. Insérer 5-10 données de test
4. Écrire 5 requêtes SELECT pour répondre à des questions métier
5. Définir les droits d'accès (qui peut faire quoi ?)
6. Proposer une stratégie de sauvegarde (fréquence, lieu)

# Synthèse et auto-évaluation

## Êtes-vous capable de...?

- ✓ Transformer un besoin métier en schéma relationnel ?
- ✓ Créer des tables avec clés primaires et étrangères ?
- ✓ Insérer et modifier des données ?
- ✓ Écrire des requêtes SELECT avec filtres et tris ?
- ✓ Utiliser agrégations et jointures ?
- ✓ Gérer les utilisateurs et les permissions ?
- ✓ Expliquer intégrité, confidentialité et disponibilité ?
- ✓ Proposer une stratégie de sauvegarde ?

## Points clés à retenir

- Base de données = données structurées, persistantes, cohérentes, partagées, interrogeables
- Modèle relationnel = tables, attributs, tuples, clés primaires/étrangères
- SQL = langage pour créer (DDL), modifier (DML), interroger (DQL) les données
- Sécurité = authentification, autorisation, intégrité, confidentialité, disponibilité
- Administration = création, maintenance, sauvegarde, monitoring, récupération